Distributed Systems · Monsoon 2026 · Course project, Option 1

# Chord vs Pastry

Study notes and viva questions for the mid-semester presentation. Everything we built, why we built it, and how to answer the hard questions.

Prateek Goyal · Abhinav Bachchas · Devansh Pandey

Papers: Stoica et al., Chord (SIGCOMM 2001, IEEE/ACM ToN 2003) and Rowstron & Druschel, Pastry (Middleware 2001)

# How to use this guide

## Part 1

Understand it. The problem, both papers, the related work, our code and our results, from zero. Read it once, fully.

## Part 2

Practise it. 124 questions the professor may ask, with short answers you can say out loud.

## Part 3

Remember it. One page of numbers and a glossary to read just before the presentation.

**Rules for the viva.** Answer in two or three sentences. Give a number when you have one. Point to a slide or a function name when it helps. If something is not done or not tested yet, say so plainly: “we have not tested that yet, it is planned for Phase 5”. A clear honest answer is worth more than a long guess.

### Who should read what

| Presenter | Slides | Must know very well |
|---|---|---|
| Speaker A | Problem, motivation, summary (slides 1–3 and 14–15) | Sections 1.1–1.2, questions 1–32 |
| Speaker B | Both papers and related work (slides 4–9) | Sections 1.3–1.6, questions 33–86 |
| Speaker C | Our work and results (slides 10–13) | Sections 1.7–1.9, questions 87–124 |

Everyone should read the whole guide once. The professor can ask anyone anything.

# Contents

**Part 1 · Understand it**

1.1 The problem 4

1.2 Distributed hash tables and consistent hashing 5

1.3 Paper 1: Chord 6

1.4 Paper 2: Pastry 10

1.5 Chord and Pastry side by side 14

1.6 Related and recent work 14

1.7 Our implementation 18

1.8 Our results 22

1.9 Limits and next steps 26

**Part 2 · Questions and answers** 27

**Part 3 · Numbers to remember and glossary** 44

Part 1 · Understand it

# 1.1 The problem

Thousands of computers each store some files. There is no central server. If you know the name of a file, how do you find the computer that has it?

This is called the **lookup problem** in peer-to-peer (P2P) systems. In a P2P system every computer (a **node** or **peer**) is equal: it stores data, asks for data and helps others find data. Computers keep joining, leaving and crashing. This constant change is called **churn**.

### Three simple ideas, and why they fail

| Idea | Example | Problem |
|---|---|---|
| One central directory | Napster | Single point of failure. One server must handle every request, so it becomes a bottleneck and can be shut down. |
| Ask everyone (flooding) | Gnutella | No memory needed, but a search sends huge numbers of messages and can still miss rare files. |
| Everyone knows everyone | a full table on each node | One hop per search, but each node remembers N others, and every join or crash must update all N nodes. |

### What a good solution needs

- **No central server.** Nothing that can crash and take the system down.
- **Small memory.** Each node remembers only a few other nodes, about log N.
- **Few steps.** Any key is found in about log N messages.
- **Keeps working** while nodes join, leave and crash.

A **distributed hash table (DHT)** meets all four. Chord and Pastry are two famous DHT designs from 2001. With a million nodes, $\log_2 N$ is only 20, so both memory and search length stay small.

**What we study.** We implement Chord and Pastry from their original papers, run both on the same simulated network, and compare hops, lookup delay, memory per node, join cost, behaviour under failures, and load balance. Our hypothesis: Pastry is faster when network delays differ a lot, because it picks nearby nodes; Chord is simpler and cheaper to keep correct.

# 1.2 Distributed hash tables and consistent hashing

A DHT works like a normal hash table, but the table is spread over many computers.

1. **Hash everything into one number space.** Every node and every key is turned into a number with a hash function. Both papers use SHA-1, which gives 160-bit numbers that look random and spread evenly.
2. **Fix one rule for ownership.** Chord: a key belongs to the first node at or after it on a ring (its **successor**). Pastry: a key belongs to the node with the **numerically closest** ID.
3. **Anyone can find the owner** by following the rule. Nobody keeps a list of who has what.

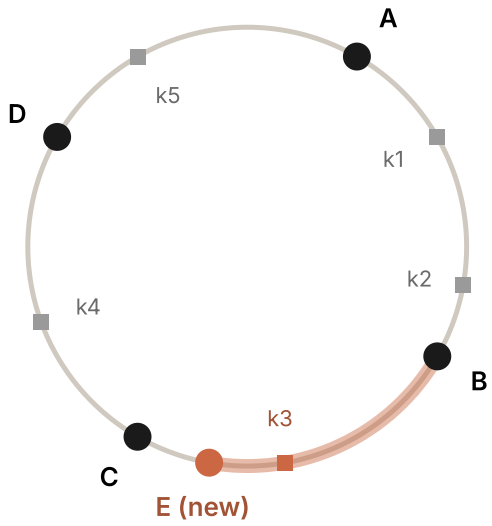



Figure 1. Consistent hashing. Each key (square) belongs to the first node clockwise. When the new node E joins between B and C, only key k3 on the orange arc moves (from C to E). Every other key stays where it was.

### Why not simply use hash(key) mod N?

With mod N, adding one server changes N, so almost every key gets a new home. Our Phase 0 demo measured it: with 10 servers, adding one moves **91%** of the keys with mod N, but only **9%** with consistent hashing (average over many random rings; the theory says 1/(N+1) = 9.1%). On a ring only the arc next to the new node changes owner.

**Load balance.** With one random point per node, some nodes get much bigger arcs than others. The Chord paper fixes this with **virtual nodes**: each real node takes several IDs. We have not added virtual nodes; this will show up when we measure keys per node.

# 1.3 Paper 1: Chord

| | |
|---|---|
| Paper | Chord: A Scalable Peer-to-peer Lookup Service for Internet Applications |
| Authors | Ion Stoica, Robert Morris, David Karger, M. Frans Kaashoek, Hari Balakrishnan (MIT; Stoica later at UC Berkeley) |
| Venue | ACM SIGCOMM 2001; longer version in IEEE/ACM Transactions on Networking, 2003. Both are top venues in networking. |
| Recognition | SIGCOMM Test of Time award, 2011 (shared with the CAN paper) |
| In one sentence | Put nodes and keys on a ring; store each key at its successor; keep shortcuts to nodes ½, ¼, ⅛ … of the way round, so every lookup takes about log N hops. |

## The architecture

IDs are m-bit numbers placed on a circle from 0 to $2^m - 1$. Each node stores three things:

- **Successor** (and a **successor list** of the next r nodes) and **predecessor**: its neighbours on the ring.
- **Finger table** with m entries: finger[i] = the first node at or after $(n + 2^i)$. So the fingers point ½, ¼, ⅛ … of the way round. Only about $\log_2 N$ of them are different nodes.

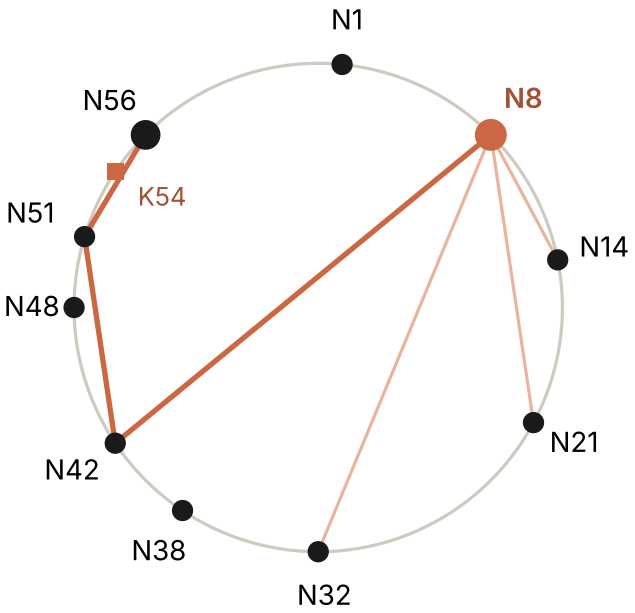



Figure 2. Redrawn from Fig. 4 of the Chord paper (m = 6, so 64 IDs). Light lines: node N8's fingers. Thick orange line: the lookup for key 54 going N8 → N42 → N51 → N56. Key K54 is stored at its successor, N56.

| i | finger start = 8 + $2^i$ | finger[i] = first node at or after it |
|---|---|---|
| 0, 1, 2 | 9, 10, 12 | N14 |
| 3 | 16 | N21 |
| 4 | 24 | N32 |
| 5 | 40 | N42 |

# How a lookup works

```
n.find_successor(id):
    if id in (n, successor]:
        return successor                # my successor owns it
    n2 = closest_preceding_node(id)
    return n2.find_successor(id)        # ask a node closer to the key

n.closest_preceding_node(id):
    for i = m down to 1:                # biggest jump first
        if finger[i] in (n, id):
            return finger[i]
    return n
```

**Walk-through (Figure 2).** N8 wants key 54. 54 is not between 8 and 14, so N8 jumps to its farthest finger before 54: N42. N42's fingers are N48, N51 and N1/N14; the farthest one before 54 is N51. For N51, 54 is between 51 and its successor 56, so the answer is N56. Three hops.

### Why O(log N) hops?

The farthest finger that does not pass the key always covers at least half of the remaining distance. Halving the distance again and again reaches the key's predecessor in about log N steps (the paper's Theorem IV.2, "with high probability").

### Why about ½·$\log_2$N on average?

The paper explains it like this: write the distance from the node to the key in binary. Each 1-bit needs one finger jump; each 0-bit needs none. A random number has about half its bits set, so the average is about ½·$\log_2$N (paper Fig. 10: path length is about ½·$\log_2$N).

# Joining, stabilization and failures

**Join.** A new node n only needs one existing node n′. It asks n′ to find successor(n) and sets that as its successor. That's all; the rest of the ring learns about n slowly, through stabilization.

**Stabilize** runs on every node about once a second:

```
n.stabilize():
    x = successor.predecessor
    if x in (n, successor):     # someone joined between us
        successor = x
    successor.notify(n)

n.notify(n2):                   # n2 thinks it might be my predecessor
    if predecessor is nil or n2 in (predecessor, n):
        predecessor = n2
```

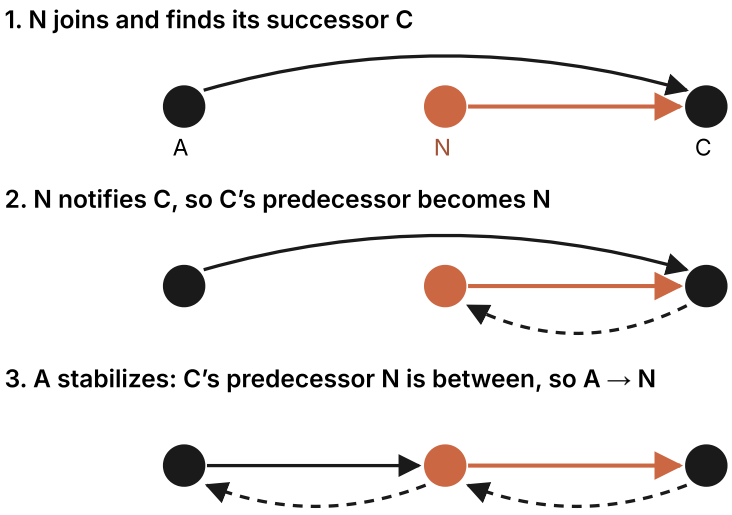



Figure 3. How stabilize links in a new node N between A and C. Solid arrow: successor pointer. Dashed arrow: predecessor pointer. After step 3 the ring A → N → C is correct.

- **fix_fingers** refreshes one finger at a time, so fingers slowly learn about new nodes.
- **check_predecessor** clears the predecessor if it stops answering.
- **Successor list.** Each node keeps its next r successors. If the first one crashes, it uses the next. The ring only breaks if all r fail at once; if each node fails with probability p, that has probability $p^r$.
- **Voluntary leave.** A leaving node gives its keys to its successor and tells its predecessor and successor about each other.

**Key idea to remember.** Correctness only needs correct **successor pointers**. Fingers only make lookups fast. A wrong finger costs extra hops but never gives a wrong answer.

# What the paper proves

| Theorem | In plain words |
|---|---|
| IV.2 | A lookup contacts O(log N) nodes, with high probability. |
| IV.3 | If nodes join while stabilize runs, the successor pointers eventually form one correct ring. |
| IV.4 | Even if up to N new nodes join a stable ring and only successors are correct, lookups still take O(log N). |
| IV.5 | With a successor list of length O(log N), even if every node fails with probability ½, lookups still find the closest living successor. |
| IV.6 | In that same failure case, a lookup still takes O(log N) expected time. |

# What the paper measured

- A simulator that runs lookups in the **iterative** style (the asking node contacts each hop itself). The paper notes Chord can also be run recursively.
- **Load balance:** with $10^4$ nodes, keys per node vary a lot; virtual nodes reduce this.
- **Path length** (Fig. 10): grows like $\frac{1}{2}\cdot\log_2 N$, about 6 hops for $2^{12}$ nodes.
- **Failures** (Table II): 1,000 nodes, successor list r = 20. With no failures the mean path length is **3.84**; with 50% of nodes failed it is 5.09 hops with about 5 timeouts per lookup.

# What is good and what is weak

**Good**

- Simple: one ring and one rule.
- Small state: about log N fingers.
- Proven bounds on hops and repair.
- Correctness depends only on successors.

**Weak**

- Ignores physical distance: every hop can cross the world.
- Uneven load without virtual nodes.
- Zave (2012) showed the original protocol can break under some failure orders.
- No defence against malicious nodes.

# 1.4 Paper 2: Pastry

| | |
|---|---|
| Paper | Pastry: Scalable, decentralized object location and routing for large-scale peer-to-peer systems |
| Authors | Antony Rowstron (Microsoft Research, Cambridge) and Peter Druschel (Rice University) |
| Venue | IFIP/ACM Middleware 2001, the main conference for middleware and distributed systems platforms |
| Recognition | Middleware 10-Year Best Paper award, 2011 |
| In one sentence | Route by matching one more digit of the key at each hop, finish with a leaf set of numerically close nodes, and fill the routing table with nearby nodes so hops are short. |

## The architecture: what one node keeps

Nodelds are 128-bit numbers on a circle, read as digits in base $2^b$. With b = 4 a digit is one hex character, so an ID has 32 digits. A key is stored at the live node whose ID is **numerically closest** to it. Each node keeps three tables (the paper's Fig. 1 shows all three for one node):

| row | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | a | b | c | d | e | f |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0: _ | 0… | 1… | 2… | 3… | 4… | 5… | 6… | 7… | 8… | 9… | a… | b… | c… | me | e… | f… |
| 1: d_ | d0 | d1 | d2 | d3 | me | d5 | d6 | d7 | d8 | d9 | da | db | dc | dd | de | df |
| 2: d4_ | d40 | · | d42 | · | d44 | d45 | me | · | d48 | · | d4a | · | d4c | · | d4e | · |

Figure 4. The routing table of a node whose ID starts with d46a1c. Row l, column c holds some node whose ID shares my first l digits and has c as the next digit. The orange cell is my own digit (no entry needed). Dots mean no such node exists yet. Lower rows get emptier, so only about $\log_{16}$N rows are filled.

| Table | Size (b = 4) | What it is for |
|---|---|---|
| **Routing table** | about 15 × $\log_{16}$N entries | Normal hops: each hop moves to a node sharing one more digit with the key. Among all nodes that fit an entry, keep the one with the lowest delay. |
| **Leaf set L** | 16 (8 smaller, 8 larger IDs) | The numerically closest nodes. Used for the last hop and for correctness: if the key is within its range, the closest leaf is the owner. |
| **Neighbourhood set M** | 32 | The nodes with the lowest delay, whatever their ID. Not used for normal routing; used to keep the routing table local (and in the rare case). |

# How routing works (the paper's Table 1)

```
route(key D) at node A:
    if D is inside the range of my leaf set:
        forward to the leaf whose ID is closest to D   (maybe me: then I am the owner)
    else:
        l = number of starting digits D and A share
        if routing table R[l][digit l of D] exists:
            forward to it                              # one more digit matches
        else:                                          # the "rare case"
            forward to any known node T with
                shared prefix(T, D) ⩾ l  and
                |T - D| < |A - D|                      # numerically closer
```




Figure 5. Our own example route for key d46a1c starting at node 65a1fc. Each hop shares one more starting digit with the key (orange). The last hop comes from the leaf set: d467c4 is the closest ID to the key.

### Why about $\log_{16} N$ hops?

Each hop fixes at least one more hex digit. Only about $N/16^k$ nodes share k digits with a given key, so after about $\log_{16} N$ hops the message is among the key's numerical neighbours and the leaf set finishes it. The paper: Pastry routes to the closest node in less than $\lceil \log_{2^b} N \rceil$ steps in normal operation, and delivery is guaranteed unless $\lfloor |L|/2 \rfloor$ nodes with neighbouring IDs fail at the same time (8 with $|L| = 16$).

# Joining

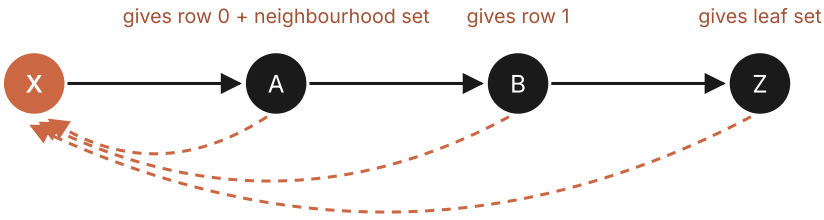



Figure 6. A new node X joins through a nearby node A. The join message, with key = X's own ID, travels A → B → Z (black arrows); each node sends state back to X (orange). The i-th node on the route gives X row i of its routing table; A gives its neighbourhood set; Z, the node numerically closest to X, gives its leaf set.

1. X must know a **nearby** Pastry node A. The paper assumes X already knows one; how to find it is outside the paper.
2. X asks A to route a "join" message with key = X's own ID. It reaches Z, the existing node numerically closest to X.
3. Nodes on the route send X their state: row i from the i-th node, the neighbourhood set from A, the leaf set from Z.
4. X can optionally improve its table: it asks the nodes in its table and neighbourhood set for their state and keeps any closer nodes (a second locality step the paper evaluates in Fig. 7).
5. X sends its state to every node in its tables, so they can add X.

The paper puts the join cost at $O(\log_{2^b} N)$ messages, with a large constant because whole rows and sets are copied.

# Locality: why Pastry is fast

Many nodes can fill an entry in row 0 (any node with that first digit), fewer in row 1, and so on. Pastry keeps the one with the lowest **proximity metric** (for example network delay or IP hops). So the first hops are very short, and only the last one or two hops may go far. The paper shows (Fig. 6) that Pastry's route distance stays close to that of an ideal "complete routing table".

# Failures and repair

- **Leaf set:** neighbours exchange keep-alive messages. If a leaf fails, ask the live leaf with the largest index on that side for its leaf set.
- **Routing table:** repaired lazily. When an entry R[l][d] is found dead, ask another node in row l for its entry R[l][d]; if that does not help, ask nodes in row l + 1.
- **Neighbourhood set:** probed from time to time; a dead member is replaced using other members' sets.

# The Pastry API (what an application sees)

`pastryInit` joins the network, `route(msg, key)` sends a message to the key's owner, and the application gets upcalls: `deliver` (the message arrived at the owner), `forward` (a message passes through this node) and `newLeafs` (the leaf set changed). Our code has `route`, `deliver` and `on_visit` (like `forward`).

# What the paper measured

- Pastry was written in **Java** and tested on an **emulated network of up to 100,000 nodes**, with b = 4, $|L|$ = 16, $|M|$ = 32 and 200,000 lookups per experiment.
- **Hops** (Fig. 4) grow like $\log_{16}N$ and stay below it. The hop distribution at 100,000 nodes (Fig. 5) has 64% of routes taking 4 hops; the average is about 4.0 ($\log_{16}N$ = 4.15).
- **Failures** (Fig. 10): 5,000 nodes, 500 fail. Average hops 2.73 before, 2.96 after the failures without repair, 2.74 after repair. Repair needed 57 remote calls per failed node.

# What is good and what is weak

**Good**

- Fewer hops: about $\log_{16}N$.
- Low delay thanks to nearby table entries.
- Leaf set gives strong correctness.
- Many choices per entry give flexibility.

**Weak**

- More state: table + 16 leaves + 32 nearby nodes.
- More complex join and repair.
- Needs delay measurements and a nearby bootstrap node.
- Table entries can stay stale (lazy repair). No defence against malicious nodes in this paper.

# 1.5 Chord and Pastry side by side

| | Chord | Pastry |
|---|---|---|
| ID space | m-bit ring (paper uses SHA-1, 160 bits) | 128-bit circle, read as base-$2^b$ digits |
| Owner of a key | Successor: first node clockwise | Node with the numerically closest ID |
| Next hop | Farthest finger that does not pass the key | Node sharing one more digit with the key |
| State per node | About $\log_2 N$ fingers + r successors + predecessor | About $15 \cdot \log_{16} N$ table entries + 16 leaves + 32 neighbours |
| Expected hops | About $\frac{1}{2} \cdot \log_2 N$ | About $\log_{16} N$ (= $\frac{1}{4} \cdot \log_2 N$) |
| Uses physical distance | No | Yes, prefers nearby nodes |
| What keeps it correct | Successor pointers | Leaf sets |
| Repair | Periodic stabilize, fix_fingers, check_predecessor | Leaf-set keep-alives, lazy table repair |
| Join cost (papers) | $O(\log^2 N)$ messages | $O(\log_2{}^b N)$ messages, large constant |
| Implemented in (papers) | Simulator with iterative lookups | Java, emulated network up to 100,000 nodes |

**The trade-off in one line.** Pastry spends more memory and more complex maintenance to get fewer, shorter hops. Chord keeps less state and simpler repair but each hop can cross the whole network.

# 1.6 Related and recent work

Our two main papers are from 2001. These papers place them in context. The professor asked for at least three recent papers; we review four in detail (2003, 2012/2017, 2021, 2022).

| Year | Paper | Venue | Idea, in one line |
|---|---|---|---|
| 2001 | Ratnasamy et al., CAN | ACM SIGCOMM | Nodes own zones of a d-dimensional space; about d·N^(1/d) hops. |
| 2002 | Maymounkov, Mazières, Kademlia | IPTPS | Distance = XOR of IDs; buckets of k contacts; parallel lookups. Used in BitTorrent, IPFS, Ethereum. |
| 2003 | Gummadi et al., DHT routing geometry | ACM SIGCOMM | Reviewed below. |
| 2004 | Zhao et al., Tapestry | IEEE JSAC | Prefix routing like Pastry, with locality. |
| 2012, 2017 | Zave, making Chord correct | ACM CCR; IEEE TSE | Reviewed below. |
| 2021 | Kanemitsu, Nakazato, KadRTT | IFIP Networking | Reviewed below. |
| 2022 | Trautwein et al., IPFS | ACM SIGCOMM | Reviewed below. |

# Review 1 · The impact of DHT routing geometry on resilience and proximity

K. Gummadi, R. Gummadi, S. Gribble, S. Ratnasamy, S. Shenker, I. Stoica · ACM SIGCOMM 2003

**What they did.** They separated a DHT's *geometry* (the shape of the graph used for routing) from its algorithm and compared five geometries: tree, hypercube, butterfly, ring and XOR. They measured **static resilience** (how many lookups still succeed right after random failures, before any repair) and **proximity** (how well nodes can choose nearby neighbours and next hops).

**What they found.** The ring is the most **flexible** geometry, and flexibility, meaning freedom to choose neighbours and next hops, gives the best resilience and the best proximity performance.

**What it means for us.** Chord is a ring; Pastry's prefix table is tree-like with a ring of leaves. Their result says a ring could also use nearby nodes (choose a finger anywhere in its range), but original Chord does not. So part of the latency gap we measure comes from Chord not using locality, not from the ring itself. We will test the resilience claim with our crash experiments.

# Review 2 · Is Chord correct?

P. Zave · "Using lightweight modeling to understand Chord", ACM SIGCOMM CCR 2012 · "Reasoning about identifier spaces: How to make Chord correct", IEEE Transactions on Software Engineering 2017

**What she did.** Wrote Chord's ring-maintenance protocol as a small formal model in Alloy, a tool that automatically checks every possible sequence of events up to a small size.

**What she found.** Under the same assumptions as the Chord papers, no published version of Chord is fully correct: some orders of joins and failures leave the ring broken in a way stabilize cannot fix. In 2017 she gave a corrected specification (including how to initialise the ring), an invariant that must always hold, and two proofs (one informal for any size, one automatic for bounded size).

**What it means for us.** Our Chord follows the original papers, so it can have the same rare problems even though all our tests pass. Random tests can find bugs but cannot prove there are none. We plan to run her problem cases on our code.

# Review 3 · KadRTT: routing with network proximity in Kademlia

H. Kanemitsu, H. Nakazato · IFIP Networking 2021

**What they did.** Improved Kademlia, the DHT inside IPFS (in libp2p), in two ways: (1) choose the next hop in order of round-trip time, but only if it still makes enough progress towards the key in ID distance, so the hop count does not grow; (2) arrange the IDs in each bucket evenly so lookups start closer to the key.

**What they found.** In simulation, KadRTT had lower lookup delay and fewer hops than the other Kademlia-based DHTs they compared.

**What it means for us.** This is the same trade-off Pastry made in 2001: prefer nearby nodes without adding hops. Our comparison of Pastry with “nearby” vs “random” tables measures exactly this effect.

## Review 4 · Design and evaluation of IPFS

D. Trautwein, A. Raman, G. Tyson, I. Castro, W. Scott, M. Schubotz, B. Gipp, Y. Psaras · ACM SIGCOMM 2022

**What they did.** Described IPFS, an open peer-to-peer network for storing and delivering content, which finds content through a Kademlia DHT, and measured the live network.

**What they found.** Peers in more than 2,700 autonomous systems (separate networks such as ISPs) in 152 countries, most of them outside big cloud providers; millions of content retrievals per day; publishing and retrieval delays acceptable for many uses.

**What it means for us.** DHTs work at real scale today, and in real networks nodes come and go all the time. That is why our next phase adds churn instead of testing only stable networks.

# 1.7 Our implementation

We wrote Chord and Pastry ourselves from the papers, in Python, inside a simulator we also wrote. No DHT library and no code from GitHub.

## Why a simulation?

- **Repeatable:** every random choice comes from one seed, so the same seed gives exactly the same numbers.
- **Big:** thousands of nodes on one laptop. Real machines would need a large testbed.
- **Controlled:** we set the delays, message loss and crashes ourselves, so we can test one thing at a time.
- **Fair:** both algorithms run on exactly the same network.

The papers did the same: Chord was evaluated mainly in a simulator, and Pastry on an emulated network in one Java process.

## Architecture of our code

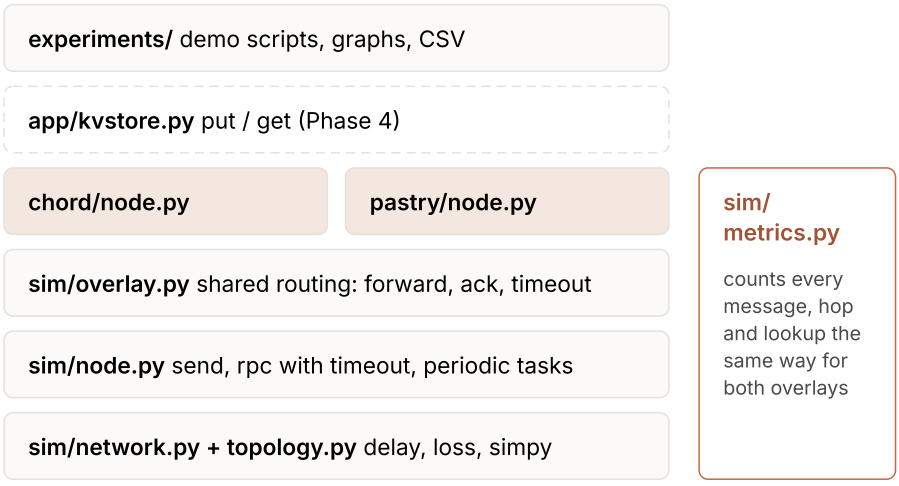



Figure 7. Our code, from experiments at the top to the simulated network at the bottom. Chord and Pastry share the same routing layer and the same metrics, so they are measured the same way. The dashed box is Phase 4.

| File | What it does |
|---|---|
| `sim/rng.py`, `sim/ids.py` | Seeded random numbers; SHA-1 hashing; the ring interval check "is x between a and b clockwise". |
| `sim/topology.py` | Where nodes sit on a 2D map: evenly spread, clustered ("continents"), or all-equal delay (control). |
| `sim/network.py` | Delivers a message after delay = distance (+ optional jitter), drops some messages (loss), drops messages to dead nodes. |
| `sim/node.py` | Base node: `send`, `rpc` (wait for a reply or time out), message handlers, periodic tasks, `fail()`. |
| `sim/overlay.py` | Routing shared by both: forward hop by hop, wait for an ack, skip dead next hops, send the answer back. |
| `chord/node.py`, `pastry/node.py` | The two algorithms. Each mainly supplies `next_hop()`, `join()`, repair and leave. |
| `chord/ring.py`, `pastry/ring.py` | Test helpers only: a checker ("oracle") that knows the real owner of every key, and builders that fill perfect tables fast. |

# How one lookup travels

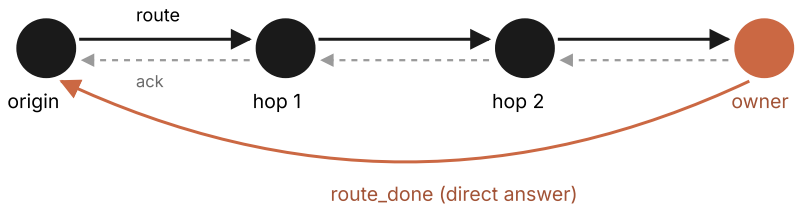



Figure 8. Recursive routing, the same for Chord and Pastry. Each node forwards the request and gets a quick ack; if no ack comes in 500 ms, it treats the next hop as dead and tries another. The owner answers the origin directly. Hops = forwards until the owner. Delay = the forward path plus the direct reply.

# How we kept the comparison fair

- Same routing code, acks, timeouts and hop counting for both (`sim/overlay.py`).
- Same metrics collector: the network itself counts every message.
- Same node positions for both, by using the same seed.
- Three maps: **uniform** (100 × 100 ms square, average distance 52 ms), **clustered** (5 "continents"), and **homogeneous** (every link 50 ms) as a control where locality cannot help.

# What we changed from the papers, and why

| Chord change | Why |
|---|---|
| IDs of 32 bits, not 160 | Fewer fingers to refresh, faster runs. Hop counts do not depend on m. ID clashes are checked and re-hashed. |
| Recursive routing with acks | The same mechanism as Pastry, so delays are compared fairly. (The paper's simulator was iterative.) |
| Owner check (pred, n] | A node answers lookups for its own keys in 0 hops, like Pastry. |
| closest_preceding_node also looks at the successor list | Suggested in the longer paper; helps route around failed fingers. |
| Join fills fingers but does not update other nodes' fingers | Other nodes find the new node through their own fix_fingers, as in the longer paper. |
| Successor list r = 8, timers about 1 s (randomised) | Enough for our sizes; random timers stop all nodes acting at the same moment. |

| Pastry change | Why |
|---|---|
| Delay read from the simulator | Simple and exact. A real node would ping. This helps Pastry a little; we say so. |
| Bootstrap = the nearest node | The paper assumes the new node knows a nearby node. |
| Join state carried inside the join message | Fewer separate messages; the same information. |
| Our own keep-alive rule | The paper does not fix one. Each second: swap leaf sets with the two direct neighbours, ping one other leaf and one neighbourhood node. |
| Ping before adding a node heard about from others | Fixes the bug below. |
| Graceful leave added | The paper treats every departure as a failure; we need leave() for the shared interface. |

### The bug we found in our Pastry, and how we fixed it

1. **What we saw.** After crashing 25% of 200 nodes, 540 dead entries were still in leaf sets 60 seconds later, and the number was not going down.
2. **Why.** A node removes a dead neighbour, then copies a neighbour's leaf set that still lists the dead node (that neighbour has not noticed yet), and the dead node is back. Old information spreads faster than detection removes it. Chord does not have this problem because stabilize replaces the successor list with a fresh copy.

3. **Fix.** A node heard about second-hand must answer a ping before it enters the leaf set, and recently failed nodes are remembered for 30 seconds. After the fix, leaf sets were fully correct within 60 seconds.

## How we know the code is correct

- An **oracle** (test-only code that sees all nodes) knows the true owner of every key. Every lookup in every experiment is checked: **100% correct** in all stable and crash runs so far.
- **116 pytest tests**: exact message delays, timeouts, every one of 256 IDs on a small Chord ring, joins at the same time, leaves, crashes, Pastry's rare case, locality, same seed gives same result.
- **Planted bugs.** We broke the code on purpose (for example, Chord jumping to the nearest finger instead of the farthest) and checked that tests fail. One planted bug was not caught at first, so we added a test for it.

## Tools

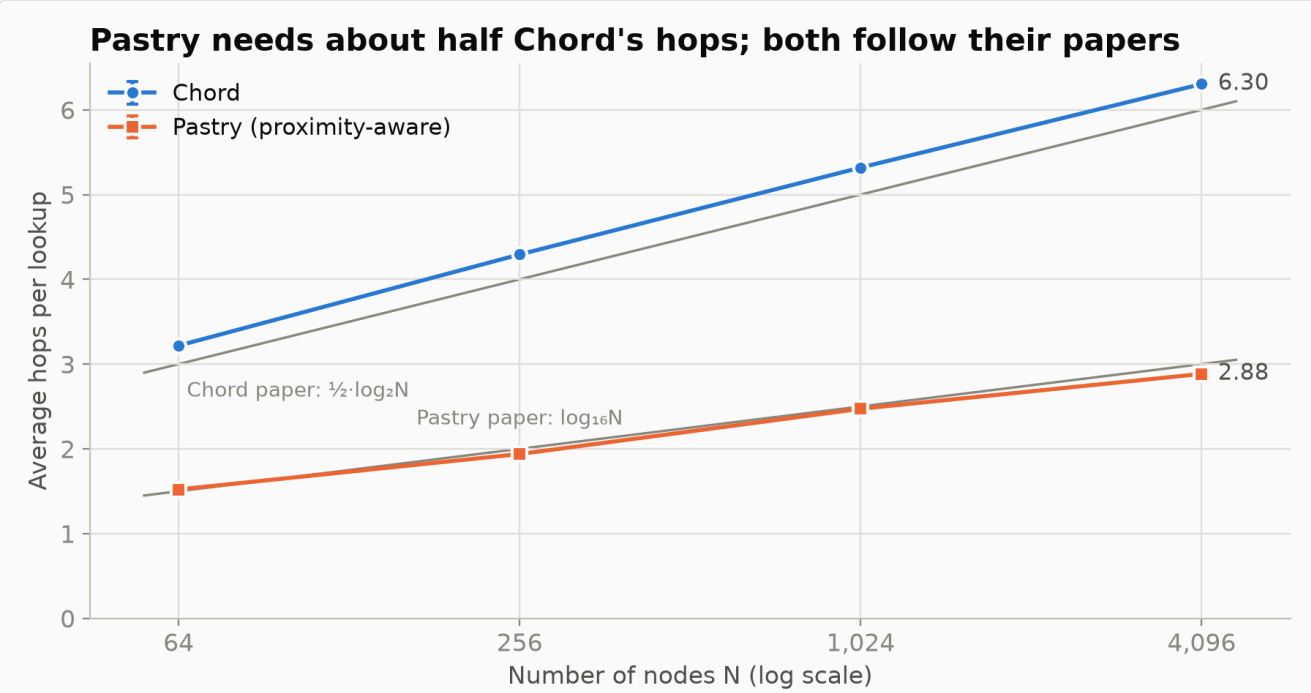
Python 3.14 · simpy 4.1 (discrete-event simulation) · hashlib SHA-1 · numpy, pandas, matplotlib (results and graphs) · pytest (tests) · Git and GitHub.

```
python -m pytest                          # 116 tests
python -m experiments.phase2_chord_demo   # Chord numbers
python -m experiments.phase3_pastry_demo  # Pastry numbers
python -m experiments.midsem_graphs       # the five graphs
```

# 1.8 Our results

Stable networks, 3 seeds, the same node positions for both overlays. Error bars are one standard deviation across seeds.

## Result 1 · Hops follow the papers

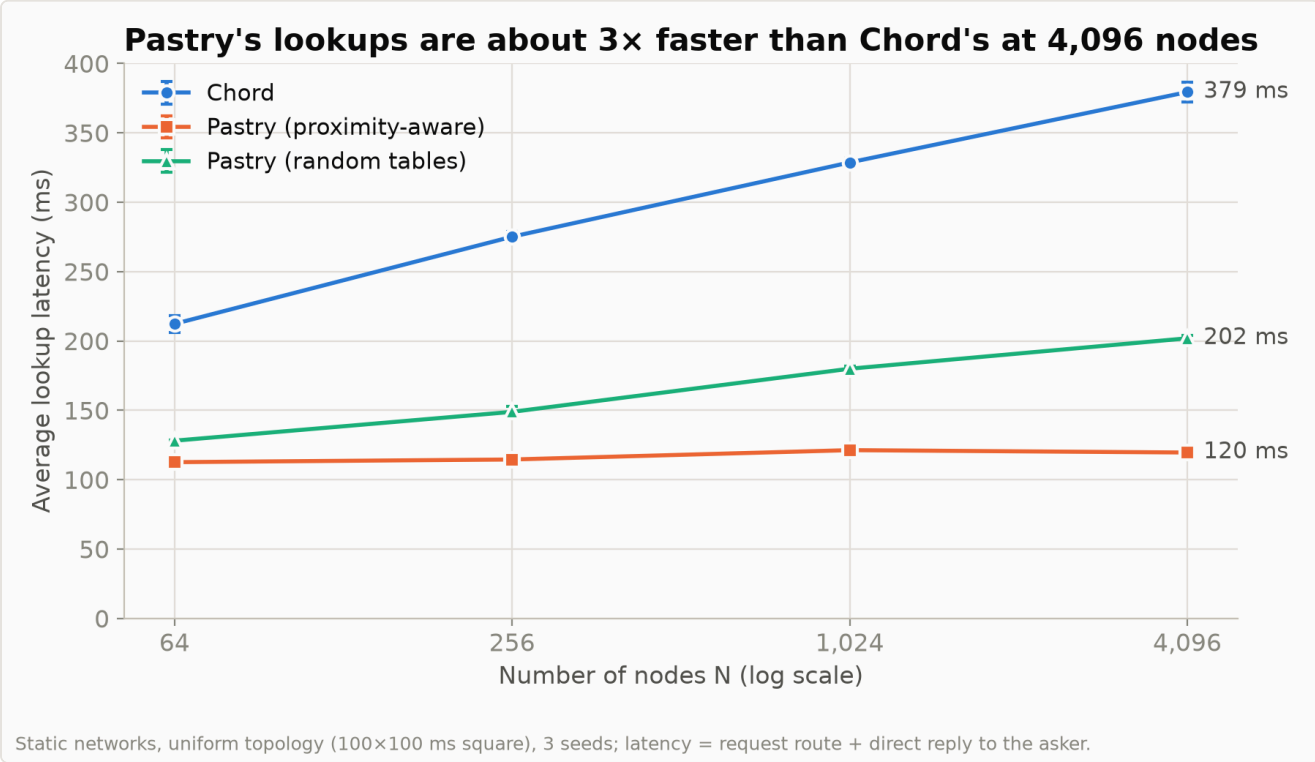



Static networks, uniform topology, 3 seeds; error bars ±1 std across seeds. Hops counted until the lookup reaches the key's owner.

At 4,096 nodes Chord needs 6.30 hops ($\frac{1}{2}\cdot\log_2 N = 6.0$) and Pastry 2.88 ($\log_{16} N = 3.0$). Chord gains ½ hop each time N doubles; Pastry gains 1 hop each time N grows 16 times.

Chord is about 0.3 hops above $\frac{1}{2}\cdot\log_2 N$ because we count the last hop from the key's predecessor to the owner. With the paper's own setting (1,000 nodes, 20 successors) and the paper's way of counting, we get **3.72** hops against the paper's **3.84**.

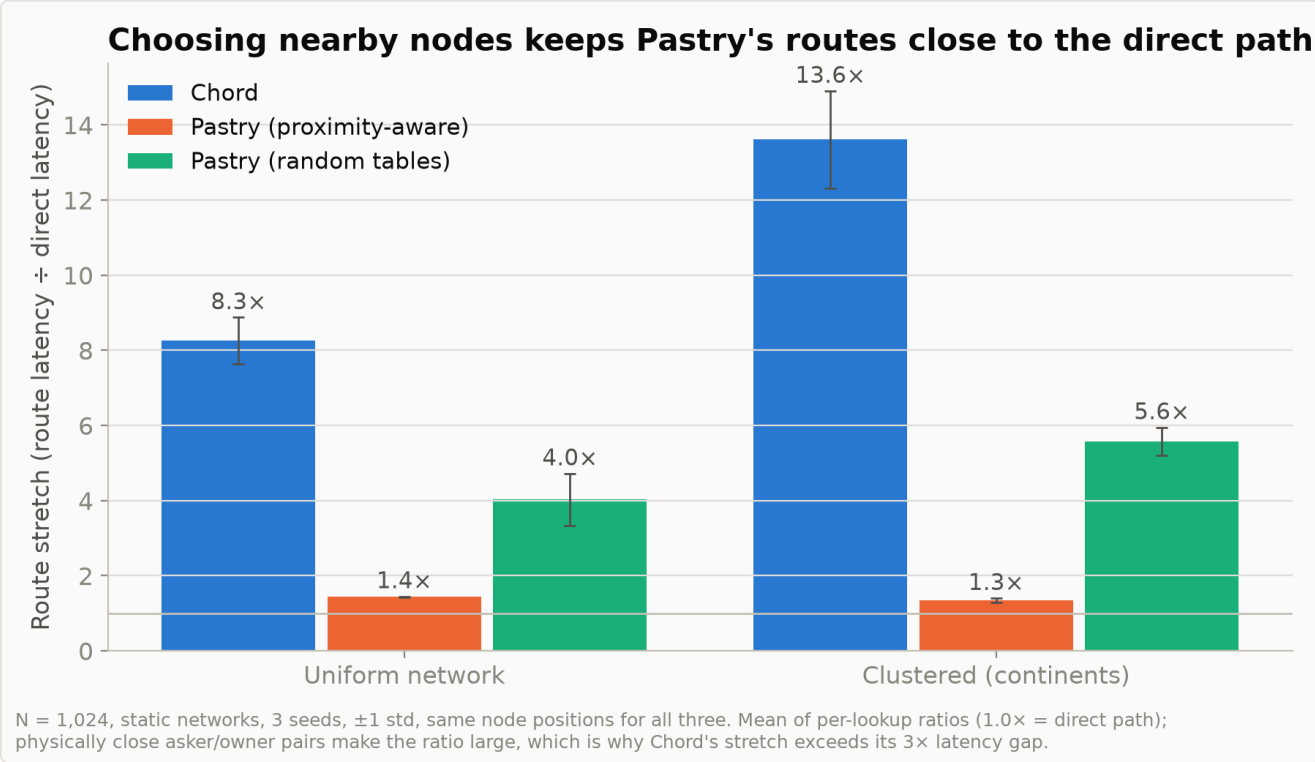
# Result 2 · Pastry finds data about 3× faster




At 4,096 nodes: Chord 379 ms, Pastry 120 ms, Pastry with random tables 202 ms. Fewer hops explain part of the gap; choosing nearby nodes explains the rest. Pastry’s delay hardly grows with N.

Chord’s delay is almost exactly (hops + 1) × 52 ms, and 52 ms is the average distance between two random points on our map. So every Chord hop goes to a random place.

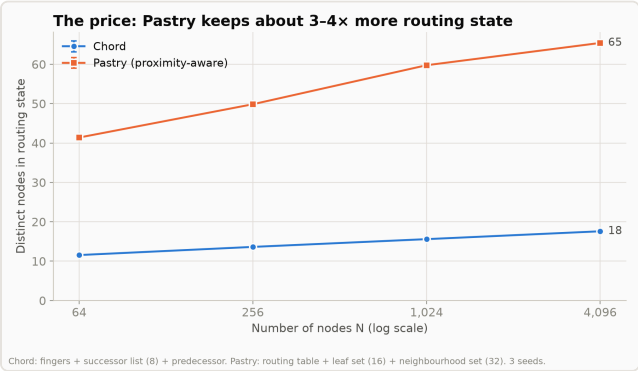
# Result 3 · Nearby nodes keep routes short



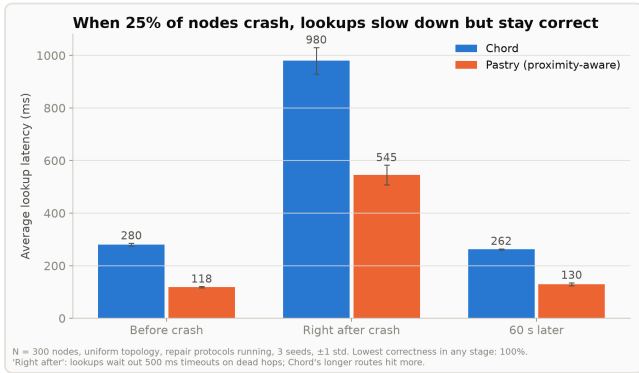

Stretch = route delay ÷ direct delay. Pastry with nearby tables: 1.4× (uniform) and 1.3× (clustered). The same Pastry with random tables: 4.0× and 5.6×, with the same hop count. Chord: 8.3× and 13.6×.

Why is Chord's stretch (8×) bigger than its 3× delay gap? Stretch is averaged per lookup. When the asking node and the owner happen to be close, the direct delay is tiny, so the ratio becomes huge. Delay is the fairer headline number; stretch shows the locality effect.

# Result 4 · The price of Pastry, and crashes




Pastry keeps 65 nodes, Chord 18 (at 4,096 nodes).




25% of 300 nodes crash. Chord 280 → 980 → 262 ms; Pastry 118 → 545 → 130 ms. 100% correct throughout.

## More numbers from our runs

| What | Chord | Pastry |
|---|---|---|
| Messages per join | About 104 (1,000 nodes) | 125 with the refine step, 50 without (300 nodes) |
| Time to fully repair after 25% crash | Ring and fingers correct in 30–60 s | Leaf sets 60 s; neighbourhood sets 2–4 min; table entries only when used (152 stale after 4 min) |
| Join-built tables vs perfect tables | Converge to the perfect state | 99% complete, entries 4% slower than the best possible |
| Message loss 10% | 19% of RPCs time out, as theory says ($1 - 0.9^2 = 19\%$) | |

## Our numbers against the papers

| Setting in the paper | Paper | Ours |
|---|---|---|
| Chord, hops vs N (Fig. 10) | about $\frac{1}{2} \cdot \log_2 N$ (6 at 4,096) | 6.30, counting to the owner |
| Chord, 1,000 nodes, r = 20 (Table II) | 3.84 | 3.72 with the same setting |
| Pastry, hops vs N (Fig. 4, 5) | below $\log_{16} N$; about 4.0 at 100,000 | 2.88 at 4,096 ($\log_{16} N = 3.0$) |
| Pastry, 500 of 5,000 fail (Fig. 10) | 2.73 → 2.96 → 2.74 hops | 25% crash: 100% correct, repaired |
| Pastry repair cost | 57 messages per failed node | not measured yet |

# 1.9 Limits and next steps

## Honest limits

- **No churn yet.** All results so far use stable networks or one big crash. Real systems have nodes joining and leaving all the time.
- **Free delay information.** Our Pastry reads delays from the simulator instead of measuring them with pings. This makes its tables a bit better than real Pastry's and costs no messages.
- **Simple network model.** Delay = straight-line distance. No bandwidth limits, queues or CPU time. Real Internet delays do not always follow the triangle rule.
- **Few seeds.** Three seeds per setting so far.
- **Zave's cases not tested.** Our Chord follows the original protocol and may have the rare problems she found.
- **No virtual nodes**, so load balance will look uneven.

## Next steps

| Phase | Plan |
|---|---|
| Phase 4 | Key-value store (put, get) on both overlays; move keys on join and leave; optional copies on successors / leaf set; churn generator. |
| Phase 5 | Full experiments: more seeds, the equal-delay control, churn rates like those measured in real systems, repair cost, keys per node, Zave's problem cases. |
| Phase 6 | Analysis and the IEEE-format report. |
| Final | Second presentation with the working proof of concept. |

# Questions he may ask

Short answers you can say out loud. Lines with an orange bar are for when he pushes further.

## A · Basics

**1 What is a peer-to-peer system?**

A system where every computer is both a client and a server. There is no central server; the computers (peers) store data and help each other find it. BitTorrent and IPFS are examples.

**2 What is an overlay network?**

A logical network built on top of the Internet. A link means "I know this node's address", and the algorithm decides who links to whom, not the cables. One overlay hop can cross many Internet routers.

**3 What is a distributed hash table?**

A hash table spread over many computers. It answers one question: which node is responsible for this key? On top of that you can build put(key, value) and get(key).

> The core operation is lookup(key) → node. Storage is an application built on top of it; that is our Phase 4.

**4 What does a DHT give you that a normal hash table does not?**

It has no single machine that holds everything, so it scales to many machines, has no central point of failure, and keeps working when some machines leave or crash.

**5 What is a hash function, and why SHA-1?**

A function that turns any input into a fixed-size number that looks random. Both papers use SHA-1 because it spreads IDs evenly. We only need even spreading, not security.

> SHA-1 is broken for security, but that does not matter here. If attackers can choose their own IDs, any hash function has the same problem (a Sybil attack). Security is outside our scope.

**6 What is consistent hashing?**

Put nodes and keys on the same ring. Each key belongs to the first node clockwise. When a node joins or leaves, only the keys next to it move, about K/N of them.

**7 Why not just use hash(key) mod N?**

When N changes, almost every key moves. In our demo, adding one server to 10 moved 91% of keys with mod N and only 9% with consistent hashing.

### 8 What does O(log N) mean here, and why is it good?

Doubling the number of nodes adds only a constant number of hops. With a million nodes, $\log_2 N$ is about 20. Knowing only your neighbour gives N hops; knowing everyone gives 1 hop but N memory and very costly updates. O(log N) for both memory and hops is the balance.

### 9 What is a hop?

One forwarding of the request from one node to the next.

### 10 Are fewer hops always faster?

No. A hop can take 2 ms or 200 ms depending on where the next node is. That is why we measure delay as well as hops. Chord and Pastry differ most on this.

### 11 What is churn?

Nodes continuously joining, leaving and crashing. Real P2P systems have a lot of it.

### 12 What is the difference between structured and unstructured P2P?

In structured P2P (Chord, Pastry) the links follow the IDs, so a lookup is guaranteed to finish in a bounded number of hops. Unstructured P2P (Gnutella) floods or randomly searches and gives no guarantee.

### 13 What is the difference between iterative and recursive lookup?

Iterative: the asking node contacts every hop itself and is told the next one. Recursive: each node forwards the request, and the owner answers. Recursive has lower delay. We use recursive for both overlays.

### 14 What does “with high probability” mean in the papers?

Node IDs are random, so a bad layout is possible but very unlikely. The bound holds except with a tiny probability that gets smaller as N grows.

### 15 What are latency, round-trip time and stretch?

Latency is the time for a message to go one way. Round-trip time is there and back. Stretch is route delay divided by the delay of sending directly to the destination; 1.0 means no detour cost.

### 16 What is routing state?

The number of other nodes a node must remember to route (fingers, successor list, leaf set, and so on). More state means more to keep up to date.

### 17 What is load balance, and what are virtual nodes?

Load balance is how evenly keys are spread over nodes. With one random ID per node, some nodes own much bigger arcs. Virtual nodes give each real node several IDs, which evens it out.

### 18 What is a successor and a predecessor?

The next node clockwise on the ring and the previous one. In Chord a key is stored at its successor.

### 19 What is the difference between lookup and get?

Lookup finds which node owns the key. Get asks that node for the value. Our results so far are lookups; get and put come in Phase 4.

### 20 What is a failure detector here?

A timeout. If a node does not answer within the time limit, we treat it as dead. It can be wrong: a slow node or a lost message looks the same as a crash.

## B · Motivation and "why"

### 21 Why did you choose this project?

Chord and Pastry solve the same problem in very different ways: a ring with shortcuts against prefix matching with locality. A fair side-by-side test shows which design choices really matter: fewer hops, nearby nodes, or simple repair.

### 22 What is the benefit of doing this?

Each paper evaluated itself in its own setup, so their numbers cannot be compared directly. We run both on the same network with the same measurements. That tells you, for a given network, which design gives lower delay and what it costs in memory and repair. We also end up with a reusable simulator to test changes.

### 23 These papers are from 2001. Why study them now?

They are the foundations of systems in use today: Dynamo and Cassandra use consistent hashing rings; BitTorrent, IPFS and Ethereum use a DHT (Kademlia). Research on them continues: Zave showed in 2012 and 2017 that Chord was not fully correct, KadRTT (2021) improves locality in Kademlia, and the IPFS paper (2022) measures a DHT at scale.

### 24 Why Chord and Pastry and not Kademlia?

They are the two classic designs from the same year with the most different ideas, both won 10-year awards, and both are in the textbooks. Kademlia would be a good third algorithm if time allows.

### 25 Why Option 1?

Option 1 asks us to implement two related algorithms and compare their performance with an application. That is exactly this project: two DHT lookup algorithms, a key-value store application on top, and a comparison.

### 26 Are these algorithms covered in our syllabus?

No. In Kshemkalyani and Singhal they are in Chapter 18 (P2P computing), and in Ghosh in Chapter 21 (social and P2P networks). Neither chapter is on the course's list of chapters to avoid. We checked both books' contents.

### 27 What is your hypothesis?

Pastry has lower lookup delay when network delays differ a lot, because it chooses nearby nodes. Chord is simpler and cheaper to maintain because it keeps less state.

### 28 How do you test it?

We measure delay on three maps. On the evenly spread and clustered maps Pastry should win. On the all-equal-delay map, locality cannot help, so any remaining gap comes only from hop count. For maintenance we measure state size, join messages and how long repair takes after crashes.

### 29 What is the "application" required by Option 1?

A key-value store with put and get built on both overlays (Phase 4), driven by experiment scripts that generate lookups, joins, leaves and crashes and record the results.

### 30 What is new in your work if both algorithms already exist?

The algorithms are not new, which is what Option 1 asks for. What we add: both built from the papers and tested on identical conditions, a control map that separates the hop effect from the locality effect, a check of our numbers against the papers' numbers, and a maintenance problem we found and fixed in our Pastry.

### 31 Who would use your results?

Anyone choosing an overlay for a system: if delay matters and nodes are spread over continents, Pastry-like locality is worth its extra state; if nodes are close together or memory and simplicity matter more, a ring like Chord is enough.

### 32 What did you learn from the project so far?

That hops and delay are different things, that correctness of these protocols rests on one small structure (successors or leaf sets), and that maintenance details, like copying stale lists, can quietly break a protocol.

# C · The Chord paper

### 33 Explain Chord in one minute.

Nodes and keys are hashed onto a ring. A key is stored at its successor, the first node clockwise. Each node keeps a finger table: finger i is the first node at least $2^i$ positions ahead. To find a key, jump to the farthest finger that does not pass the key; each jump at least halves the distance, so it takes O(log N) hops. Stabilize runs every second to fix links after joins, and a successor list handles crashes.

### 34 What is a finger table? Give an example.

m entries; finger[i] = successor(n + $2^i$). In the paper's example (64 IDs), node 8 has fingers N14, N14, N14, N21, N32, N42 for starts 9, 10, 12, 16, 24, 40.

### 35 Why does finger i point $2^i$ places ahead?

So that from any node there is a finger covering at least half of the remaining distance to any key. Each hop halves the distance, which gives log N hops.

### 36 Why is the average ½·$\log_2 N$ and not $\log_2 N$?

Write the distance to the key in binary. Each 1-bit needs a finger jump and each 0-bit is skipped. About half the bits are 1, so about ½·$\log_2 N$ jumps.

### 37 Why "closest preceding" and not "closest"?

We must never jump past the key. The node just before the key (its predecessor) is the one that knows the owner, which is its successor.

### 38 What happens if a finger is wrong or dead?

A wrong finger only costs extra hops. A dead finger causes a timeout and we skip to another. Lookups stay correct as long as successor pointers are right.

### 39 What do stabilize and notify do?

Stabilize asks my successor for its predecessor x. If x is between me and my successor, a node has joined there, so x becomes my successor. Then I notify my successor that I might be its predecessor. Notify accepts me only if I am between its current predecessor and itself.

> The "only if between" check stops an old or far-away node from replacing a good predecessor. We found our convergence tests did not catch a bug in this check, so we added a direct test.

### 40 How does a node join Chord?

It asks any existing node to find its successor and sets it. It copies the successor's list and fills its fingers. Stabilize and notify then link it in fully; other nodes find it through their fix_fingers.

### 41 What if many nodes join at the same time?

Pointers can be wrong for a while, but the paper proves (Theorem IV.3) that with stabilize running the successors eventually form one correct ring. We tested 30 nodes joining at the same moment; the ring became fully correct.

### 42 Why a successor list, and how long should it be?

If my successor crashes I can use the next one. The ring breaks only if all r successors fail together, which has probability $p^r$. The paper suggests r = O(log N); its experiment uses r = 20 = $2 \cdot \log_2 N$ for 1,000 nodes. We use r = 8.

### 43 What happens when a node crashes?

It just stops. Others notice by timeouts: stabilize skips a dead successor, check_predecessor clears a dead predecessor, and lookups skip dead fingers. The successor list keeps the ring connected.

### 44 What does a voluntary leave do?

The leaving node gives its keys to its successor, tells its successor about its predecessor and tells its predecessor about its successor list, so the neighbours are fixed at once.

### 45 When a node joins, which keys move?

The keys between its predecessor and itself, which were stored at its successor. Only about K/N keys.

### 46 How much does a join cost?

The paper: $O(\log^2 N)$ messages. We measured about 104 messages per join into a 1,000-node ring: about log N finger lookups, each about ½·log N hops.

### 47 What does Theorem IV.5 say?

With a successor list of length O(log N), even if every node fails with probability one half, lookups still find the closest living successor, with high probability.

### 48 Did the Chord paper use recursive or iterative lookups?

Its simulator was iterative; the paper says Chord can be run either way. We use recursive for both overlays so delays are fair.

### 49 What did the Chord experiments show?

Path length grows like $\frac{1}{2} \cdot \log_2 N$ (about 6 hops for $2^{12}$ nodes). Keys per node vary a lot without virtual nodes. With 1,000 nodes and r = 20, the mean path is 3.84 hops; with half the nodes failed it rises to about 5 hops with about 5 timeouts per lookup.

### 50 What are Chord's weaknesses?

It ignores physical distance, so each hop can cross the network. Load is uneven without virtual nodes. Zave showed the original protocol is not correct for some failure sequences. It has no defence against malicious nodes.

---

### 51 Could Chord use locality too?

Yes. Finger i can be any node in the range $[n + 2^i, n + 2^{i+1})$, so Chord could pick the closest one in that range. Gummadi et al. (2003) showed the ring is flexible enough for this. The original paper does not do it, and neither does our Chord.

---

### 52 What are m and r in your code?

m = 32-bit IDs (the paper uses 160) and r = 8 successors. Hop counts do not depend on m; we check that no two nodes get the same ID.

---

# D · The Pastry paper

**53 Explain Pastry in one minute.**

IDs are 128-bit numbers read as hex digits. A key belongs to the node with the numerically closest ID. Each hop goes to a node that shares one more starting digit with the key, so it takes about $\log_{16}N$ hops. The last hop uses the leaf set, the 16 nodes with the closest IDs. For each routing table entry Pastry keeps the node with the lowest delay, so most hops are short.

---

**54 What is b and how does it change things?**

Digits are in base $2^b$. A larger b gives fewer hops ($\log_{2^b}N$) but a bigger table ($2^b - 1$ entries per row). The paper uses b = 4, so hex digits, 15 entries per row.

> With b = 1 digits are bits, and Pastry needs about $\log_2 N$ hops, similar to Chord.

---

**55 What is in row l, column c of the routing table?**

A node whose ID shares my first l digits and has c as the next digit. Row 0 covers the first digit, row 1 the second, and so on.

---

**56 Why is one column in each row empty?**

It is my own digit at that position. A node with that digit would share one more digit with me, so it belongs in the next row.

---

**57 What is the leaf set for?**

It holds the $|L|$ numerically closest nodes, half smaller and half larger. If a key is inside its range, the closest leaf is the owner. It handles the last hop and is what makes routing correct.

---

**58 What is the neighbourhood set for?**

It holds the $|M|$ nodes with the lowest delay, whatever their IDs. It is not used for normal routing. It helps keep the routing table full of nearby nodes, especially when a node joins.

---

**59 Walk me through the routing algorithm.**

If the key is within my leaf set range, send it to the closest leaf. Otherwise find l, the number of digits the key shares with me, and use table entry R[l][next digit of the key]. If that entry is empty, the rare case: send it to any known node that shares at least l digits with the key and is numerically closer than me.

---

**60 When does the rare case happen?**

When the needed table entry is empty or its node is dead. With full tables it is rare. In our tests, lookups stayed 100% correct even with 70% of table entries removed.

---

### 61 Why does Pastry routing always finish?

Every hop either shares more digits with the key or keeps the same prefix and gets numerically closer. Neither can go on forever.

### 62 Why about $\log_{16} N$ hops?

Each hop fixes one hex digit. Only about $N/16^k$ nodes share k digits with the key, so after about $\log_{16} N$ hops you are among the key's close neighbours and the leaf set finishes.

### 63 When can Pastry fail to deliver?

The paper says delivery is guaranteed unless $\lfloor |L|/2 \rfloor$ nodes with neighbouring IDs fail at the same time; that is 8 with $|L| = 16$.

### 64 How does a node join Pastry?

The new node X asks a nearby node A to route a join message to X's own ID. It reaches Z, the node closest to X. The i-th node on the path gives X row i of its table, A gives its neighbourhood set, and Z gives its leaf set. Then X tells all nodes in its tables that it has arrived.

### 65 Why should the first contact node be nearby?

X copies A's neighbourhood set and early rows. If A is close to X, those nodes are also close to X, so X's table starts with good, nearby entries.

### 66 What is the proximity metric?

A measure of distance in the real network, such as delay or the number of IP hops. Pastry assumes each node can measure it to any other node. In our simulator it is the delay on our map.

### 67 Why are Pastry's early hops short?

Row 0 entries only need the right first digit, so about N/16 nodes qualify and Pastry can pick a very close one. Deeper rows have fewer choices, so later hops are longer, but there are only one or two of them.

### 68 How does Pastry repair failures?

Leaf set: ask the farthest live leaf on that side for its leaf set. Routing table: lazily, when an entry is found dead, ask other nodes in the same row (then the next row) for a replacement. Neighbourhood set: probe members and replace dead ones from others' sets.

### 69 What is the Pastry API?

pastryInit to join, route(msg, key) to send, and upcalls to the application: deliver at the owner, forward at each node on the way, and newLeafs when the leaf set changes.

### 70 What did the Pastry experiments show?

Written in Java, run on an emulated network of up to 100,000 nodes. Hops grow like $\log_{16} N$ and stay below it (about 4.0 at 100,000). With 500 of 5,000 nodes failing, hops went 2.73 → 2.96 → 2.74 after repair, which needed 57 messages per failed node. Routes stayed close to an ideal complete table in distance.

---

### 71 What are Pastry's weaknesses?

More state and more complex join and repair. It needs delay measurements and a nearby bootstrap node. Table entries can stay stale because repair is lazy. No defence against malicious nodes in this paper.

---

### 72 What is the second join step and is it worth it?

After the first join, X asks every node in its table and neighbourhood for their state and keeps closer nodes. In our runs it cost 2.5× more join messages (125 vs 50) for entries about 1% closer, with no visible delay gain. Our first step already finds near-best entries.

---

# E · Comparing the two

### 73 So which one is better?

It depends on what matters. For lookup delay on a network where delays vary, Pastry: about 3× faster in our tests. For memory and simple maintenance, Chord: about 18 nodes remembered against 65, and faster clean-up after crashes.

### 74 Why does Pastry need fewer hops?

Each Pastry hop fixes one hex digit, which is 4 bits. A Chord hop fixes one bit of the distance, and only about half the bits need fixing. So Chord needs about ½·$\log_2 N$ hops and Pastry about ¼·$\log_2 N$.

### 75 Why does Pastry keep more state?

Its table has up to 15 entries per filled row, plus 16 leaves and 32 neighbours. Chord has about $\log_2 N$ different fingers and a short successor list.

### 76 What do the two have in common?

Both hash nodes and keys into one ID space, both need O(log N) hops and O(log N) state, both repair themselves, and both have one small structure that guarantees correctness: successors in Chord, the leaf set in Pastry.

### 77 Which is easier to implement?

Chord. Its whole protocol is a few short functions. Pastry's join and repair have more parts, and the maintenance bug we found was in Pastry.

### 78 Is the comparison fair if Pastry uses b = 4?

b = 4 is the value the paper uses and recommends, so it is a fair comparison of the designs as published. We can also run Pastry with other b values.

### 79 How do they compare with Kademlia?

Kademlia uses XOR of IDs as distance, keeps buckets of k contacts, and asks several nodes in parallel. It also gives log N hops and can prefer nearby contacts. It is the design used in BitTorrent, IPFS and Ethereum today.

### 80 Why did Chord's delay grow with N but Pastry's stayed flat?

Chord's hops grow with N and each hop is a random distance. Pastry's extra hops are in the early rows, where it can choose very close nodes, so they add almost no delay.

# F · Related work

**81 Name three recent papers related to your work.**

Zave, "How to make Chord correct", IEEE TSE 2017; Kanemitsu and Nakazato, "KadRTT", IFIP Networking 2021; Trautwein et al., "Design and evaluation of IPFS", ACM SIGCOMM 2022. We also review Gummadi et al., SIGCOMM 2003, which compares routing shapes.

**82 What did Gummadi et al. find?**

They compared tree, hypercube, butterfly, ring and XOR geometries. The ring is the most flexible, and flexibility gives the best resilience to failures and the best use of nearby nodes. So a ring like Chord could use locality too; original Chord just does not.

**83 What did Zave show? Does that mean your Chord is wrong?**

She modelled Chord in Alloy and showed that under the paper's own assumptions no published version is fully correct: some orders of joins and failures break the ring. In 2017 she published a fixed version with proofs. Our Chord follows the original, so it may have the same rare problems. Our tests pass, but tests cannot prove correctness. We plan to run her cases.

**84 What is KadRTT?**

An improved Kademlia that chooses next hops by lowest round-trip time, but only if they still make enough progress toward the key, and spreads IDs evenly in each bucket. In simulation it reduced both delay and hops. It is Pastry's locality idea applied to today's DHT.

**85 What does the IPFS paper tell you?**

A DHT runs at real scale: IPFS peers are in more than 2,700 networks in 152 countries, with millions of retrievals a day. It also shows nodes come and go a lot, which is why we add churn next.

**86 What other DHTs do you know?**

CAN (zones in a d-dimensional space), Tapestry (prefix routing like Pastry), Kademlia (XOR distance), and Koorde (based on de Bruijn graphs, in Ghosh Chapter 21).

# G · Our implementation

**87 Why Python?**

It is quick to write and easy to read, and simpy gives us a simulation clock. We compare algorithms by hops, delay and messages inside the simulation, so the speed of Python itself does not affect the results.

### 88 What is simpy, and what is a discrete-event simulation?

simpy keeps a list of future events, like “message arrives at time 12.3 ms”, and jumps from one event to the next. Nothing happens between events, so we can simulate minutes of network time in seconds.

### 89 What does yield do in your code?

In simpy a process is a Python generator. “yield event” pauses the process until that event happens in simulated time. For example, rpc() sends a request and yields until the reply or the timeout comes.

### 90 How is message delay calculated?

Every node has an (x, y) position. Delay = the straight-line distance in milliseconds, plus optional random jitter. Some messages can be dropped (loss), and messages to dead nodes are dropped.

### 91 How do nodes detect failures?

By timeouts. Every forward waits for an ack; if none comes within 500 ms, the next hop is treated as dead, removed from the tables, and another hop is tried.

### 92 How do you count hops and measure delay?

Hops = number of forwards until the request reaches the owner (0 if the asking node owns the key). Delay = simulated time from sending the lookup until the answer comes back. A test checks that delay equals the sum of the link delays on the path plus the direct reply.

### 93 How did you make the comparison fair?

Same routing, ack and timeout code for both; the same metrics code; the same node positions (same seed); and three maps, including an all-equal-delay control.

### 94 What is the oracle? Isn’t it cheating?

It is test-only code that sees all nodes and knows the true owner of every key. Nodes never use it. It only checks afterwards whether each lookup answer was right.

### 95 What is the static builder and why use it?

It fills every node’s tables directly with the correct values, as if the network had been stable for a long time. Simulating thousands of joins is slow, so for large-N lookup tests we build statically. Joins are tested separately. For Pastry, tables built by real joins gave almost the same delay (102.8 ms vs 102.3 ms).

### 96 What is “Pastry with random tables”?

The same Pastry with each table entry chosen at random among valid nodes instead of the nearest one. Same hops, no locality. It separates the effect of fewer hops from the effect of nearby nodes.

### 97 What are your three maps and why the third one?

Evenly spread nodes; clustered nodes like continents; and every link 50 ms. On the third map picking nearby nodes cannot help, so it shows how much of Pastry's advantage comes from hops alone.

### 98 What bug did you find?

In our Pastry, after many crashes, dead nodes kept coming back into leaf sets because nodes copied each other's old leaf sets. We fixed it by pinging a node heard about second-hand before adding it, and remembering recent failures for 30 seconds. After the fix, leaf sets were correct within 60 seconds.

### 99 How do you know the code is correct?

Every lookup is checked by the oracle: 100% correct in all our runs. 116 pytest tests check delays, timeouts, every ID on a small ring, joins, leaves and crashes. We also planted bugs on purpose to check that the tests catch them.

### 100 What did you change from the papers?

Chord: 32-bit IDs, recursive routing, an extra check so a node answers its own keys, successor list of 8. Pastry: delay read from the simulator, the nearest node as bootstrap, our own keep-alive rule, ping before adding a second-hand node, and a graceful leave. All listed in the slides and the guide.

# H · Results

**101 Summarise your results in three sentences.**

Both overlays match their papers: Chord about ½·$\log_2 N$ hops, Pastry about $\log_{16} N$. Pastry is about 3× faster (120 vs 379 ms at 4,096 nodes) because of fewer hops and nearby nodes. Chord keeps far less state (18 vs 65) and repairs itself faster after crashes.

**102 Your Chord gives 6.30 hops but ½·$\log_2 N$ is 6.0. Why?**

We count the last hop from the key's predecessor to the owner; the paper's count stops at the predecessor. With the paper's own setting (1,000 nodes, 20 successors) and its way of counting, we get 3.72 against the paper's 3.84.

**103 Why is Chord's stretch 8× when its delay is only 3× Pastry's?**

Stretch is averaged per lookup. When the asking node and the owner happen to be close, the direct delay is tiny and the ratio becomes very large. Delay is the fair headline number; stretch shows the locality effect.

**104 What happened when 25% of the nodes crashed?**

Every lookup still found the right owner. Delay jumped right after the crash (Chord 980 ms, Pastry 545 ms) because lookups waited for timeouts on dead nodes. Chord suffered more because its longer routes meet more dead nodes. After 60 s both were back to normal.

**105 Are three seeds enough?**

For these averages the spread between seeds is tiny (about 0.02 hops), so the conclusions are stable. For the final results we will use more seeds and report the spread.

**106 Does your hypothesis hold?**

So far yes. Pastry is faster, especially on the clustered map, and Chord keeps less state and cleans up faster. The maintenance part is only measured roughly so far; Phase 5 measures it properly under churn.

**107 How much does a join cost in each?**

Chord about 104 messages per join into 1,000 nodes. Pastry 125 with the refine step and 50 without, into 300 nodes.

**108 What happens with message loss?**

With 10% loss, 19% of request/reply pairs time out, as expected ($1 - 0.9^2 = 19\%$). Lookups then skip and retry another hop.

# I · Tough questions and criticism

**109 Your network is simulated. Why should we trust the results?**

Our numbers match the papers' own numbers (for example 3.72 vs 3.84 hops for Chord), every lookup is checked for correctness, and the papers themselves used a simulator and an emulated network. We clearly list what the model leaves out.

**110 Your Pastry gets delay for free. Isn't that unfair to Chord?**

Yes, a little. A real Pastry node must ping to measure delay. We say this openly. The "random tables" control shows how big the locality effect is, and in Phase 5 we can add measurement noise or ping cost.

**111 What is bad about your project?**

No churn yet, a simple delay model (no bandwidth or queues), only three seeds, free delay information for Pastry, no virtual nodes, Zave's cases not tested, no security against malicious nodes, and no run on real machines.

**112 What did you do that the papers did not?**

We put both algorithms on exactly the same network, added a control map that separates hops from locality, reproduced a number from the Chord paper with its own settings, and found and fixed a leaf-set problem in our Pastry maintenance.

**113 Did you copy code or use a DHT library?**

No. We wrote both algorithms from the papers' descriptions and pseudocode; no DHT library and no code from GitHub. We can open any function and explain it.

**114 Did you use AI tools?**

Answer this honestly and simply, describing how you used them. What matters is that each of you can explain any part of the code and the results.

**115 Why not run it on real machines?**

To test thousands of nodes with controlled delays and crashes we need a simulator; real machines would give only a few nodes and noisy results. A small real version with sockets could be future work.

**116 What if a node is malicious and lies?**

Neither paper handles that, and neither do we. Later work, for example Castro et al., "Secure routing for structured peer-to-peer overlay networks" (OSDI 2002), adds defences.

### 117 What is your main contribution?

A fair, repeatable, side-by-side measurement of Chord and Pastry built from the original papers, that shows where Pastry's speed comes from (hops vs locality) and what it costs (state, join messages, repair time).

### 118 What would you add with more time?

Churn with realistic rates, Zave's failure cases, Chord with nearby fingers, Kademlia as a third algorithm, and a small run on real machines.

### 119 Why do your graphs have such small error bars?

Each point averages 1,000 lookups per seed, and the network layouts from different seeds behave very similarly at these sizes. The crash experiment has larger bars because crashes hit different nodes.

### 120 What happens if two nodes get the same ID?

With 128-bit Pastry IDs it practically never happens. With our 32-bit Chord IDs it is unlikely, and our code checks for it and re-hashes.

## J · Tools

### 121 Which tools and libraries did you use?

Python 3.14; simpy for the simulation; hashlib for SHA-1; numpy, pandas and matplotlib for numbers and graphs; pytest for tests; Git and GitHub for the code.

### 122 How do you make experiments repeatable?

Every random choice (positions, delays, crashes, lookups) comes from a random generator made from one seed plus a name. The same seed always gives exactly the same results.

### 123 How do I run your code?

Set up the virtual environment, install requirements, then run `python -m pytest` for the tests and `python -m experiments.midsem_graphs` for the graphs. The README lists all commands.

### 124 How big is the code?

About 4,000 lines of Python including tests: the simulator (sim/), Chord (chord/), Pastry (pastry/), experiment scripts and 116 tests.

Part 3 · Remember it

# Numbers to remember

**6.30 → 2.88**
hops at 4,096 nodes, Chord → Pastry

**379 → 120 ms**
lookup delay at 4,096 nodes (Pastry random tables: 202 ms)

**18 → 65**
nodes remembered per node at 4,096 nodes

**1.4× vs 4.0×**
Pastry route stretch, nearby vs random tables (uniform map)

**3.72 vs 3.84**
Chord hops, ours vs paper, 1,000 nodes, 20 successors

**91% vs 9%**
keys moved when 1 server joins 10: mod N vs ring

**980 / 545 ms**
delay right after 25% crash, Chord / Pastry; 100% correct

**≈104**
messages per Chord join into 1,000 nodes

**116**
tests; every lookup checked by the oracle

| From the papers | Value |
|---|---|
| Chord hops | about ½·$\log_2 N$ (Fig. 10); Theorem IV.2: O(log N) |
| Chord join cost | $O(\log^2 N)$ messages |
| Chord successor list | r = O(log N); experiment uses 20 for 1,000 nodes |
| Pastry hops | below $\lceil \log_{16} N \rceil$; about 4.0 at 100,000 nodes |
| Pastry parameters | b = 4, $\lvert L \rvert$ = 16, $\lvert M \rvert$ = 32 |
| Pastry delivery guarantee | unless $\lfloor \lvert L \rvert / 2 \rfloor$ = 8 neighbouring nodes fail together |
| Pastry failure test | 500 of 5,000 fail: 2.73 → 2.96 → 2.74 hops; 57 messages per failed node to repair |
| Pastry implementation | Java, emulated network up to 100,000 nodes |

# Glossary

| Term | Meaning |
|---|---|
| Node / peer | One computer taking part in the system. |
| Overlay | A logical network on top of the Internet; links are chosen by the algorithm. |
| DHT | Distributed hash table: finds the node responsible for a key, with no central server. |
| Consistent hashing | Nodes and keys on one ring; a key goes to the next node clockwise; joins move few keys. |
| Successor / predecessor | Next / previous node on the ring. |
| Finger table | Chord's shortcuts: finger i is the first node at least $2^i$ ahead. |
| Stabilize / notify | Chord's periodic repair of successor and predecessor links. |
| Successor list | The next r nodes, used when the successor crashes. |
| Routing table (Pastry) | Row l, column c: a node sharing l digits with me, then digit c. |
| Leaf set | Pastry's 16 nodes with the closest IDs; gives correctness. |
| Neighbourhood set | Pastry's 32 nodes with the lowest delay. |
| Rare case | Pastry's fallback when the table entry is missing. |
| Proximity / locality | Preferring nodes that are close in the real network. |
| Hop | One forward of a message to the next node. |
| Latency / delay | Time taken; one way for a message, end to end for a lookup. |
| Stretch | Route delay ÷ direct delay. |
| Churn | Nodes constantly joining, leaving and crashing. |
| RPC and timeout | Request that waits for a reply; no reply in time means "probably dead". |
| Recursive routing | Each node forwards the request; the owner answers the origin directly. |
| Oracle | Test-only code that knows the correct owner, used to check answers. |
| Seed | The starting number for random choices; same seed, same results. |
| Discrete-event simulation | A simulation that jumps from one event to the next (simpy). |